**SOLUCIÓN – Seguimiento 2 (B) – Unidad 2 – Computación y estructuras discretas I**

## Funciones y relaciones

Se dan las siguientes nuevas definiciones sobre relaciones (usando la relación $E = \{(a,1),(b,2),(c,3)\}$ como referencia):

- dom: *dominio de definición de una relación.* Ej: $\operatorname{dom} E = \{a,b,c\}$.
- ran: *rango de una relación.* Ej: $\operatorname{ran} E = \{1,2,3\}$.
- $R^T$: *transpuesta de una relación*, invierte las tuplas. Ej: $E^T = \{(1,a),(2,b),(3,c)\}$.
- $R_2 \circ R_1$: *composición* de $R_1$ con $R_2$. Ej: $R = \{(1,1),(1,4),(2,3),(3,1),(3,4)\}$, $S = \{(1,0),(2,0),(3,1),(3,2),(4,1)\}$, $S \circ R = \{(1,0),(1,1),(2,1),(2,2),(3,0),(3,1)\}$.

1. Tome en cuenta las siguientes relaciones $X, Y, Z$:
   - $X = \{(1,2),(2,2),(3,1),(3,3)\}$
   - $Y = \{(a,b),(d,c),(e,f),(p,w)\}$
   - $Z = \{(2,g),(1,h),(c,7),(w,9)\}$

   Basándose en las definiciones anteriores, calcule: (a) $\operatorname{ran} X$ (b) $\operatorname{ran} Z^T$ (c) $\operatorname{dom} X \cap \operatorname{dom} Y$ (d) $\operatorname{ran} Y \cup \operatorname{ran} Z$ (e) $Z \circ Y$ (f) $Z^T \circ Y^T$

(a) $\operatorname{ran} X = \{1,2,3\}$

(b) $Z^T = \{(g,2),(h,1),(7,c),(9,w)\}$, por lo tanto $\operatorname{ran} Z^T = \{1,2,c,w\}$

(c) $\operatorname{dom} X = \{1,2,3\}$ y $\operatorname{dom} Y = \{a,d,e,p\}$. Como $X$ tiene elementos numéricos e $Y$ tiene elementos simbólicos, no comparten ningún elemento.
$\therefore\ \operatorname{dom} X \cap \operatorname{dom} Y = \emptyset$

(d) $\operatorname{ran} Y = \{b,c,f,w\}$ y $\operatorname{ran} Z = \{g,h,7,9\}$ (conjuntos disjuntos).
$\therefore\ \operatorname{ran} Y \cup \operatorname{ran} Z = \{b,c,f,w,g,h,7,9\}$

(e) Se buscan pares $(a,b) \in Y$ y $(b,c) \in Z$:
$(a,b) \in Y$: necesita $(b,\cdot) \in Z \to b \notin \operatorname{dom} Z$, sin par.
$(d,c) \in Y$: necesita $(c,\cdot) \in Z \to (c,7) \in Z \Rightarrow (d,7)$.
$(e,f) \in Y$: necesita $(f,\cdot) \in Z \to f \notin \operatorname{dom} Z$, sin par.
$(p,w) \in Y$: necesita $(w,\cdot) \in Z \to (w,9) \in Z \Rightarrow (p,9)$.
$\therefore\ Z \circ Y = \{(d,7),(p,9)\}$

(f) $Y^T = \{(b,a),(c,d),(f,e),(w,p)\}$, $Z^T = \{(g,2),(h,1),(7,c),(9,w)\}$.
Se buscan pares $(a,b) \in Y^T$ y $(b,c) \in Z^T$. Los segundos elementos de $Y^T$ son $\{a,d,e,p\}$ y los primeros de $Z^T$ son $\{g,h,7,9\}$; conjuntos disjuntos, ningún par puede formarse.
$\therefore\ Z^T \circ Y^T = \emptyset$
*Nota*: Esto es consistente pues $(Z \circ Y)^T = \{(7,d),(9,p)\}$ se obtendría con $Y^T \circ Z^T$ (orden inverso), no con $Z^T \circ Y^T$.

2. Para cada función $f$ con dominio $\mathbb{R}$ (para el caso de $f_1$ se hace la exclusión $\mathbb{R} - \{-2\}$) y codominio $\mathbb{R}$, determine si es invertible o no y argumente:
   (a) $f_1(x) = \dfrac{x+1}{x+2}$
   (b) $f_2(x) = x^5 + 1$

(a) **No es invertible.** $f_1$ es inyectiva: $f_1(x_1) = f_1(x_2) \Rightarrow (x_1+1)(x_2+2) = (x_2+1)(x_1+2) \Rightarrow x_1 = x_2$. ✓
Sin embargo, *no es sobreyectiva*: el valor $y = 1$ no tiene preimagen en $\mathbb{R} - \{-2\}$, pues $\frac{x+1}{x+2} = 1 \Rightarrow x+1 = x+2 \Rightarrow 1 = 2$, contradicción. Al no ser sobreyectiva, no es biyectiva ni invertible.

(b) **Sí es invertible.** $f_2(x) = x^5 + 1$ es biyectiva:
*Inyectiva*: $x^5$ es estrictamente creciente, luego $x_1 \neq x_2 \Rightarrow x_1^5 \neq x_2^5 \Rightarrow f_2(x_1) \neq f_2(x_2)$. ✓
*Sobreyectiva*: dado cualquier $y \in \mathbb{R}$, su preimagen es $x = \sqrt[5]{y-1} \in \mathbb{R}$ (la raíz quinta está definida para todo real). ✓

## Funciones

*Nota*: Dominio $\Leftrightarrow$ codominio se indica debajo de cada función.

3. Tome en cuenta las siguientes funciones con sus respectivos dominio y codominio:

| $f(x) = -2x + 6$ | $g(x) = x^2 - 1$ | $h(x) = x$ | $m(x) = x^3 - 9x$ | $n(x) = 2x + 1$ |
|---|---|---|---|---|
| $\mathbb{R} \Leftrightarrow \mathbb{R}$ | $\mathbb{R} \Leftrightarrow \mathbb{R}$ | $\mathbb{Z} \Leftrightarrow \mathbb{R}$ | $\mathbb{R} \Leftrightarrow \mathbb{R}$ | $\mathbb{Z} \Leftrightarrow \mathbb{Z}$ |

Rellene la tabla indicando **Sí** o **No**. En caso negativo, muestre el contraejemplo.

| | ¿Es inyectiva? | ¿Es sobreyectiva? | ¿Es biyectiva? |
|---|---|---|---|
| $f(x)$ | **Sí.** $f(x_1) = f(x_2) \Rightarrow -2x_1 = -2x_2 \Rightarrow x_1 = x_2$ | **Sí.** Dado $y \in \mathbb{R}$, $x = \frac{6-y}{2} \in \mathbb{R}$ | **Sí** |
| $g(x)$ | **No.** $g(1) = g(-1) = 0$, pero $1 \neq -1$ | **No.** $\operatorname{ran}(g) = [-1, \infty)$; $-2 \notin \operatorname{ran}(g)$ | **No** |
| $h(x)$ | **Sí.** $h(x_1) = h(x_2) \Rightarrow x_1 = x_2$ | **No.** $0{,}5 \in \mathbb{R}$ pero $0{,}5 \notin \mathbb{Z}$, sin preimagen | **No** |
| $m(x)$ | **No.** $m(0) = 0$ y $m(3) = 27-27 = 0$, pero $0 \neq 3$ | **Sí.** $m$ es continua y $m(x) \to \pm\infty$, cubre todo $\mathbb{R}$ | **No** |
| $n(x)$ | **Sí.** $n(x_1) = n(x_2) \Rightarrow 2x_1 = 2x_2 \Rightarrow x_1 = x_2$ | **No.** $2 \notin \operatorname{ran}(n)$: $2x+1 = 2 \Rightarrow x = \frac{1}{2} \notin \mathbb{Z}$ | **No** |

4. Sean $f, g : \mathbb{R} \to \mathbb{R}$ definidas por $f(x) = \dfrac{x+3}{2}$ y $g(x) = 2x - 3$.
   (a) Calcule $(f \circ g)(x)$ y $(g \circ f)(x)$.
   (b) ¿Es $f \circ g = g \circ f$? Según lo visto en el curso sobre funciones compuestas e invertibles: ¿qué cree que está pasando y por qué dan dicho resultado particular?

(a) $(f \circ g)(x) = f(2x-3) = \dfrac{(2x-3)+3}{2} = \dfrac{2x}{2} = x.$ $\quad (g \circ f)(x) = g\left(\dfrac{x+3}{2}\right) = 2 \cdot \dfrac{x+3}{2} - 3 = (x+3) - 3 = x.$

(b) $g$ es la **función inversa** de $f$ (y recíprocamente): la composición en *ambos* sentidos produce la función identidad $\operatorname{id}(x) = x$, que es la condición que define a dos funciones como inversas entre sí.

(c) **Sí**, $f \circ g = g \circ f$ (ambas igualan $x$). Esto *no* ocurre para cualquier par de funciones; es exclusivo de los pares de funciones inversas entre sí. Al ser $g = f^{-1}$, cada función "deshace" la operación de la otra, recuperando la identidad independientemente del orden de composición.

---

5. **(2.50 pts) Problema práctico: Jugadores Activos — Solución**

**Contexto.** Un videojuego en línea tiene dos servidores. Cada servidor tiene su propia lista de jugadores registrados (por ID entero). El sistema central también mantiene una lista de jugadores **baneados** que no pueden participar. Se necesita obtener la lista de jugadores **activos**: todos los jugadores únicos de ambos servidores que **no** estén baneados, impresos en orden ascendente.

**Referencia de operaciones de conjunto en Java:**

| Método | Operación | Efecto sobre `setA` |
|---|---|---|
| `setA.addAll(setB)` | $A \cup B$ | Agrega a `setA` los elementos de `setB` que no estén ya presentes |
| `setA.retainAll(setB)` | $A \cap B$ | Elimina de `setA` todo lo que **no** esté en `setB` |
| `setA.removeAll(setB)` | $A \setminus B$ | Elimina de `setA` todo lo que **sí** esté en `setB` |

**Entrada:**

Primera línea: `NA NB K` con $1 \leq \text{NA} \leq 100\,000$, $1 \leq \text{NB} \leq 100\,000$, $0 \leq K \leq 50\,000$

Segunda línea: `NA` enteros — IDs servidor A

Tercera línea: `NB` enteros — IDs servidor B

Cuarta línea: `K` enteros — IDs baneados ($1 \leq \text{ID} \leq 10^9$, pueden repetirse)

**Salida:**

Primera línea: número de jugadores activos

Segunda línea: sus IDs en orden ascendente separados por espacio

**Actividad.** Implemente el método `jugadoresActivos` marcado con `TODO`. El resto del `Main.java` ya está dado y no debe modificarse.

**Ejemplos:**

*Ejemplo 1*

**Entrada:**

```
5 4 2
3 7 1 9 3
5 7 2 9
7 2
```

**Salida:**

```
4
1 3 5 9
```

*Ejemplo 2 — Todos baneados*

**Entrada:**

```
3 3 3
10 20 30
10 20 30
10 20 30
```

**Salida:**

```
0
```

*Ejemplo 3 — Baneado inexistente*

**Entrada:**

```
4 2 1
8 4 6 2
1 6
99
```

**Salida:**

```
5
1 2 4 6 8
```

**Idea de la solución.** Se construyen tres `TreeSet`: uno con los IDs del servidor A (`activos`), otro con los del servidor B (`setB`) y otro con los baneados (`setBan`). La unión $A \cup B$ se obtiene con `activos.addAll(setB)`, que incorpora los IDs de B ignorando los que ya estén presentes. Luego, la diferencia $(A \cup B) \setminus$ Baneados se obtiene con `activos.removeAll(setBan)`, que elimina en una sola llamada todos los IDs baneados que aparezcan en el conjunto. Como `activos` es un `TreeSet`, el resultado queda ordenado de forma ascendente automáticamente, sin ningún paso extra de ordenamiento.

```java
import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int na = sc.nextInt(), nb = sc.nextInt(), k = sc.nextInt();

        int[] servidorA = new int[na];
        for (int i = 0; i < na; i++) servidorA[i] = sc.nextInt();

        int[] servidorB = new int[nb];
        for (int i = 0; i < nb; i++) servidorB[i] = sc.nextInt();

        int[] baneados = new int[k];
        for (int i = 0; i < k; i++) baneados[i] = sc.nextInt();

        sc.close();

        TreeSet<Integer> activos = jugadoresActivos(servidorA, servidorB, baneados);
```

```java
21
22          System.out.println(activos.size());
23          StringBuilder sb = new StringBuilder();
24          for (int id : activos) sb.append(id).append(' ');
25          System.out.println(sb.toString().trim());
26      }
27
28      public static TreeSet<Integer> jugadoresActivos(int[] servidorA,
29                                                      int[] servidorB,
30                                                      int[] baneados) {
31          TreeSet<Integer> activos = new TreeSet<>();
32          TreeSet<Integer> setB    = new TreeSet<>();
33          TreeSet<Integer> setBan  = new TreeSet<>();
34
35          for (int x : servidorA) activos.add(x);
36          for (int x : servidorB) setB.add(x);
37          for (int x : baneados)  setBan.add(x);
38
39          activos.addAll(setB);
40          activos.removeAll(setBan);
41
42          return activos;
43      }
44  }
```

**Líneas clave:**

- **Línea 38**: `activos.addAll(setB)` — realiza la unión $A \cup B$; los duplicados entre servidores se descartan automáticamente.
- **Línea 39**: `activos.removeAll(setBan)` — realiza la diferencia $(A \cup B) \setminus \text{Baneados}$; los IDs baneados inexistentes en el conjunto se ignoran sin error.
- El uso de `TreeSet` garantiza orden ascendente sin necesidad de llamar a `Collections.sort`.